

Содержание курса.

Ниже — сжатая, согласованная программа повышения квалификации на **2 дня** → **1 неделю** → **2 недели** → **3 недели**, с упором на архитектурную строгость, надёжность и производительность. Один сквозной мини-проект эволюционирует от каркаса к прод-уровню.

Сквозной мини-проект

Сервис «Orders & Balance» на Django + PostgreSQL + GraphQL (Ariadne).

Требования по ходу программы: чёткие слои (`api` → `domain` → `infrastructure`), типичная бизнес-логика (заказ, оплата, возврат), идемпотентные мутации, аудит действий, профилируемые запросы, CQRS-read модель для отчётной витрины.

Итерация 1 — «Разбудить и собрать контекст» (2 дня)

Цель: вспомнить язык/инструменты, освежить архитектурные принципы, прогреть руки.

Результат: каркас проекта + 1–2 читаемых мутации/запроса, логирование и первые тесты.

AI генерация кода разрешена: нет.

Темы и действия

- **Python дизайн кода.** Освежить PEP 8, договорённости по именованию, типам, структуре файлов. 5 лёгких задач на LeetCode для разогрева. ([Python Enhancement Proposals \(PEPs\)](#))
- **Code-design.** Пройтись по **SOLID/GRASP** на своих примерах. Быстрый DI (через конструктор/фабрики, без магии). Мини-юнит-тесты на бизнес-правила. ([Habr](#))
- **GraphQL + Ariadne.** Схема, 1 Query + 1 Mutation, валидация, обработка ошибок. ([Habr](#))
- **PostgreSQL.** Вспомнить типы индексов и EXPLAIN на одном запросе. Коротко MVCC. ([Postgres Professional](#))
- **SQL-тренажёр.** 20 задач (джойны/агрегации): [🔍 Интерактивный тренажер по SQL — SQL Academy](#)

- **Логирование.** Включить структурированное логирование и корреляционный ID. ([Habr](#))
- **Django ORM.** `select_related/prefetch_related`, быстрый SQL-лог. ([Habr](#))
- **Архподходы (обзор).** DDD, ES, CQRS, Event Storming, идемпотентность, eventual consistency, fail-fast. ([Habr](#))

Контрольные точки

- Каркас слоёв + базовые тесты проходят.
- ГрафQL-мутация **идемпотентна** (ключ/таблица дедупликации).
- Первый `EXPLAIN ANALYZE` понят и зафиксирован. ([PostgreSQL](#))

Итерация 2 — «Вернуть рабочий темп» (1 неделя)

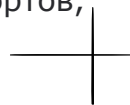
Цель: восстановить уверенное проектирование и поведение под нагрузкой.

Результат: вертикальный срез: оформленные доменные правила, оптимизированные запросы, базовая наблюдаемость.

AI генерация кода разрешена: нет.

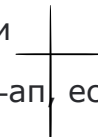
День 1–2 — Python + дизайн

- 15 задач **LeetCode (Easy/Medium)**; рефакторинг под **SRP/DIP**.
 - Шаблон проекта: изолировать доменную модель, интерфейсы портов, реализации адаптеров.
- Ресурсы: обзор SOLID/GRASP + практический DI. ([Habr](#))



День 3 — Django ORM

- Убрать N+1 (`select_related/prefetch_related`), ввести `annotate/aggregate`, `F/Case/When`. Журнал SQL и фикс-ап, если план плох. ([Habr](#))



День 4 — PostgreSQL

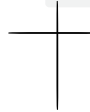
- Индексы под реальные фильтры (btree/GIN/BRIN), `EXPLAIN ANALYZE` до и после. ([Postgres Professional](#))

День 5 — GraphQL + Ariadne

- Схема: строгие типы, ошибки, пагинация/фильтрация. Хендлеры с валидацией и идемпотентностью. ([ariadnegraphql.org](#))

День 6 — Логирование

- Структурированные логи (JSON), уровни, поля: `request_id`, `user_id`, `operation`. Шаблон для RCA. ([structlog](#))



День 7 — Архподходы (прикладно)

- **Idempotency/Exactly-once**: ключ запроса + дедуп-таблица, повторяемость безопасна. ([yandex.cloud](#))
- **Eventual consistency**: где допустимо, как компенсировать. ([Habr](#))
- **Fail-fast**: немедленная ошибка, агрессивные предикаты/валидация. ([martinfowler.com](#))

Контрольные точки

- P95 ответа $\leq$ целевого (зафиксируй цель).
- Нет N+1 на «списке заказов».
- Индексы подтверждены планом.

Итерация 3 — «Инженерная глубина» (2 недели)

Цель: закрепить архитектурное мышление: надёжность, консистентность, производительность.

Результат: сервис с изолированной доменной моделью, **CQRS-read** витриной, outbox и корректной конкуренцией.

AI генерация кода разрешена: нет.

Неделя 1 — Домайн, ORM, производительность

- **Domain first**: сущности/агрегаты, инварианты, Value Objects. ([Habr](#))
- ORM-углубление: `Subquery/OuterRef`, сложные `annotate`, конкурентный апдейт `F()` без гонок. ([Habr](#))
- Профилирование SQL: `EXPLAIN ANALYZE`, горячие места, перепроектирование запроса/индекса. ([Habr](#))

Неделя 2 — Событийность и консистентность

- **CQRS**: разнести read/write-модели; материализованная витрина для отчётов. ([Habr](#))
- **Event Sourcing (минимум)**: таблица событий, rehydration, snapshotting. Версионирование событий (upcasters). ([Habr](#))

- **Outbox + Idempotency:** публикация событий транзакционно (таблица outbox + воркер) и дедуп. (yandex.cloud)
- **Distributed locks:** критичные инварианты через Redis-блокировки (понимая риски Redlock). ([Redis](https://redis.io))
- **Event Storming:** быстрая сессия для уточнения доменной ленты событий. ([Habr](https://habr.com))

Контрольные точки

- Read-витрина обновляется асинхронно, SLA чтения выдержан.
 - Переигровка событий восстанавливает состояние без ручных фиксов.
 - Блокировки устраняют двойную запись, **без** дедлоков.
-

Итерация 4 — «Прод-уровень» (3 недели)

Цель: довести проект до промышленного качества: чёткие границы, наблюдаемость, предсказуемость под нагрузкой.

Результат: полноценный модуль с графом API, устойчивостью к отказам и подтверждённой производительностью.

AI генерация кода разрешена: да.

Неделя 1 — Жёсткая архитектура

- Финализировать слои и границы модулей. Контракты интерфейсов, строгие типы.
- Полный набор юнит-тестов домена + интеграционные по API (GraphQL).
- Уточнить правила **fail-fast**: валидаторы, инварианты, ранние возвраты. (martinfowler.com)

Неделя 2 — Надёжность и консистентность

- **Idempotency/Exactly-once** на всех мутациях (ключи, видимость в логах/метриках). (yandex.cloud)
- **Saga/компенсации** для долгоживущих процессов (в простом виде — процесс-менеджер).
- Контролируемая **eventual consistency**: SLAs на задержку проекций. ([Habr](https://habr.com))

Неделя 3 — Производительность и эксплуатация

- **Postgres-тюнинг по фактам:** планы, селективность, подходящие индексы (btree/GIN/BRIN). ([Postgres Professional](https://postgrespro.com))

- **Django ORM-оптимизации:** убрать оставшиеся N+1/подзапросы, заменить на агрегации/CTE при необходимости. ([Habr](#))
- **Логирование/наблюдаемость:** структурированные логи + корреляция вызовов; подготовка шаблона RCA. ([structlog](#))
- **Анти-паттерны:** пройти чек-лист (God Object, Big Ball of Mud, Anemic Model и пр.) и устранить найденное. ([refactoring.guru](#))

Финальные критерии приёма

- P95/P99 и RPS соответствуют целям; планы запросов подтверждают выбор индексов.
 - Мутации идемпотентны, с наблюдаемой дедупликацией.
 - Read-модель обновляется и переигрывается без ручных вмешательств.
 - Код выдерживает ревью на **SRP/DI/границы контекстов**; анти-паттерны устранены.
-

Материалы (отобрано под задачи)

- **PEP 8/чистый код:** официальный PEP 8; краткие русские обзоры. ([Python Enhancement Proposals \(PEPs\)](#))
- **SOLID/GRASP:** понятные разборы на русском. ([Habr](#))
- **Pytest/юнит-тесты:** практические гайды. ([Habr](#))
- **DI в Python:** Dependency Injector, плюсы/подводные камни. ([Habr](#))
- **GraphQL + Ariadne:** вводное и API-документация. ([Habr](#))
- **PostgreSQL:** индексы/типы индексов, MVCC, EXPLAIN/ANALYZE, SELECT/INSERT/UPDATE под капотом. ([Postgres Professional](#))
- **SQL-тренажёр:**  [Интерактивный тренажер по SQL — SQL Academy](#)
- **Логирование:** best practices + структурированное (structlog). ([Habr](#))
- **Django ORM:** annotate/агрегации, борьба с N+1, русская дока. ([Habr](#))
- **DDD/ES/CQRS/Event Storming:** практичные русские разборы. ([Habr](#))
- **Idempotency/Exactly-once:** гайд и примеры. ([yandex.cloud](#))
- **Distributed locks:** Redlock/Redis docs (понимать риски). ([Redis](#))
- **Event versioning (upcasters):** эволюция схем в ES. ([Habr](#))
- **Eventual consistency:** обзор на русском. ([Habr](#))
- **Fail-fast:** статья Мартина Фаулера. ([martinfowler.com](#))

- **Анти-паттерны:** систематизированный каталог + Big Ball of Mud, Anemic Model. (refactoring.guru)
-

Мини-чек-листы (использовать в каждой итерации)

- **Архитектура:** SRP на уровне модулей; явные границы; DI без сервис-локатора.
- **Надёжность:** идемпотентные мутации; fail-fast на входе; компенсирующие действия.
- **Производительность:** нет N+1; планы запросов валидируют индексы; P95/P99 в целевых рамках.
- **Наблюдаемость:** структурированные логи с `request_id`; ошибки с контекстом; шаблон RCA.
- **Анти-паттерны:** не допускать God Object/Anemic Model/Big Ball of Mud.